

A Comprehensive Guide to Bash Job Control & Asynchronous Processing

Welcome to the ultimate guide to managing processes in Bash! This guide will take you from the fundamentals of running simple background jobs to leveraging advanced tools for interactive, asynchronous communication.

1. The Basics: What is Job Control?

In a shell, you typically run a command and wait for it to finish. This is called a **foreground process**. It occupies your terminal, and you can't do anything else until it's done.

Job control is a feature that allows you to run processes in the **background**. A background process does not block your terminal, so you can continue working while it runs. This is essential for long-running tasks like file downloads, data processing, or starting a server.

2. Simple Background Jobs: The `&` Operator

The simplest way to run a command in the background is by adding an ampersand (`&`) at the end.

```
sleep 10 &
```

When you do this, the shell will start the command and immediately return you to the prompt. It will also print the **job ID** and the **process ID (PID)**.

```
[1] 12345
```

- `[1]` is the **job ID**. This is how the shell keeps track of the job.
- `12345` is the **PID**. This is how the operating system identifies the process.

Listing Your Jobs: `jobs`

What if you forget what's running in the background? The `jobs` command lists all of your current background jobs.

```
jobs
# Output:
# [1]+  Running                  sleep 10 &
# [2]-  Running                  ping google.com &
```

Bringing Jobs to the Foreground: `fg`

If you need to interact with a background job again, you can bring it to the foreground with the `fg` command. You can refer to the job by its job ID (e.g., `%1`).

```
fg %1
```

The `sleep 10` command will now take over your terminal until it finishes.

Pausing a Foreground Job: `ctrl+z`

What if a foreground job is taking too long and you want your terminal back *without* killing the job? Press `ctrl+z`. This will **suspend** the job and put it in the background.

```
ping google.com
^Z
# Output:
# [1]+  Stopped                  ping google.com
```

Sending a Suspended Job to the Background: `bg`

The job is now paused. To make it continue running *in the background*, use the `bg` command.

```
bg %1
# Output:
# [1]+  ping google.com &
```

Now the `ping` command is running again, but in the background, leaving your terminal free.

Terminating a Job: `kill`

If you want to stop a background job for good, use the `kill` command with the job ID.

```
kill %1
# Output:
# [1]+  Terminated              ping google.com &
```

The `kill` command sends a signal to a process. By default, it sends the `SIGTERM` (15) signal, which asks the process to terminate gracefully. If a process is stuck, you can force it to shut down with the `SIGKILL` (9) signal, but this should be a last resort as it doesn't allow the process to clean up after itself.

```
kill -9 %1 # Force kill the job
kill -SIGKILL %1 # Same as above
```

Terminating by Name: `pkill` and `killall`

Sometimes it's more convenient to kill a process by its name rather than its job ID or PID. `pkill` and `killall` are perfect for this.

```
# Start a few sleep processes
sleep 1000 &
sleep 1000 &
```

```
# Kill all processes with the name 'sleep'
pkill sleep

# killall is similar, but can be more precise on some systems
killall sleep
```

Waiting for a Job to Finish: `wait`

If you have a script that starts a background job and you need to wait for it to complete before continuing, use the `wait` command.

```
echo "Starting a long process..."
sleep 5 &
pid=$! # Capture the PID of the last background command
echo "Process started with PID $pid. Waiting for it to finish."
wait $pid
echo "Process has finished."
```

3. A Practical Example: Parallel Processing

One of the most powerful uses of background jobs is to run tasks in parallel. Imagine you have a directory of images that you need to compress. Doing them one by one would be slow. With background jobs, you can do them all at once.

```
#!/bin/bash

# Create a directory for compressed images
mkdir -p ./compressed

for img in *.jpg; do
    echo "Compressing $img..."
    # Run each compression in the background
    convert "$img" -quality 75 "./compressed/$img" &
done

# The 'wait' command with no arguments waits for ALL
# background jobs started in the current shell to finish.
echo "Waiting for all compression jobs to finish..."
wait
echo "All images compressed!"
```

In this script, the `for` loop starts a `convert` process for every `.jpg` file, each running in the background. The `wait` command at the end ensures the script doesn't exit until all the parallel compression jobs are done.

4. Surviving Terminal Closure: `nohup` and `disown`

What happens to your background jobs if you close the terminal? They get terminated. To prevent this, you can use `nohup` or `disown`.

nohup : Run a Command Immune to Hangups

The `nohup` command (short for “no hangup”) runs a command that ignores the hangup signal (SIGHUP) that is sent when a terminal closes. By default, it redirects the command’s output to a file named `nohup.out`. For better control, you can redirect standard output and standard error yourself.

```
# Run a script and log all output (stdout & stderr) to my_app.log
nohup ./my_app.sh > my_app.log 2>&1 &
```

disown : Removing a Job from the Job Table

The `disown` command removes a job from the shell’s list of active jobs. This also prevents it from being killed when the shell exits.

```
sleep 100 &
jobs # Shows the sleep job
disown %1
jobs # The job is gone from the list
```

5. Advanced Interactive Processing: coproc

The methods above are great for “fire-and-forget” tasks. But what if you need to **communicate** with a background process while it’s running? This is where `coproc` shines.

`coproc` is a **Bash built-in** that runs a command asynchronously and gives you **two file descriptors** to talk to it:

- `${NAME[0]}` → read end (output from coprocess)
- `${NAME[1]}` → write end (input to coprocess)

It’s like having a **background process with a private pipe**, so you can send it commands and capture results without blocking your shell.

Syntax

```
coproc [NAME] command [redirections]
```

- `NAME` → optional variable name for the coprocess (defaults to `COPROC`).
- `command` → the command (or function/block) to run.

Minimal Example

```
coproc bc # start calculator

echo "2+3" >&${COPROC[1]} # send to bc
read result <&${COPROC[0]} # read from bc
echo "Result: $result"
```

Output:

Result: 5

Using Named Coprocs

Multiple coprocs can run at once if you give them names:

```
coproc CALC { bc -l; }
coproc PING { ping -c 3 google.com; }

echo "sqrt(144)" >&${CALC[1]}"
read val <&${CALC[0]}"
echo "Square root: $val"

while read line <&${PING[0]}; do
    echo "PING says: $line"
done
```

Patterns of Use

1. Command Filter (Interactive Pipe)

```
coproc GREP { grep "error"; }

echo "this is an error" >&${GREP[1]}"
echo "all fine" >&${GREP[1]}"

read line <&${GREP[0]}"
echo "Match: $line"
```

2. Loop Reading

```
coproc DATE { while true; do date; sleep 1; done; }

while read -t 2 line <&${DATE[0]}; do
    echo "[$(date +%T)] $line"
done

# Clean up by closing the pipes
exec {DATE[1]}>&-
exec {DATE[0]}>&-
```

whiptail & coproc for TUIs

`coproc` is fantastic for building terminal user interfaces.

```
# Gauge Progress
coproc GAUGE { whiptail --gauge "Progress..." 6 50 0; }

for i in {1..100}; do
    echo $i >&${GAUGE[1]}"
    sleep 0.05
done
```

```
exec {GAUGE[1]}>&-
```

Gotchas & Notes

- Only **one unnamed coproc** (`COPROC`) at a time. For multiple, name them.
- **Check for success:** `coproc` doesn't fail if the command can't be found. Always check if the PID was created: `[[ -n $COPROC_PID ]]` .
- **Always close file descriptors** when you're done to prevent hangs. This is critical. If the writing pipe isn't closed, the reading command (like `read` or `cat`) will wait forever.
- Coprocs are **Bash-only**, not POSIX-compliant.

6. Summary: Choosing the Right Tool

Your Goal	The Right Tool	Example
Run a simple, non-interactive background task.	<code>&</code>	<code>sleep 60 &</code>
Run many background tasks and wait for all.	<code>&</code> and <code>wait</code>	<code>for i in *; do task "\$i"</code> <code>&; done; wait</code>
Pause a foreground job and resume it later.	<code>Ctrl+Z</code> , <code>bg</code> , <code>fg</code>	<code>^Z</code> , <code>bg %1</code>
Kill a process by its job ID or PID.	<code>kill</code>	<code>kill %1</code>
Kill a process by its name.	<code>pkill</code> or <code>killall</code>	<code>pkill nginx</code>
Run a job that survives terminal closure.	<code>nohup</code> or <code>disown</code>	<code>nohup ./my_script.sh &</code>
Interact with a background process (send/read).	<code>coproc</code>	<code>coproc bc</code>
Manage multiple, persistent shell sessions.	<code>tmux</code> or <code>screen</code> (external programs)	<code>tmux new -s my_session</code>

Conclusion

You now have a powerful toolkit for managing processes in Bash. You can run simple jobs in the background, make them survive terminal closures, and even build complex, interactive applications with `coproc` . Understanding these tools will make you a more efficient and powerful command-line user.